

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 3

Контейнеризация написанного приложения средствами Docker

Выполнил:
Цатинян Артём
Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

1 Задача

Тема проекта: приложение для бронирования столиков в ресторанах.

В рамках лабораторной работы требовалось контейнеризовать микросервисное backend-приложение: реализовать `Dockerfile` для каждого сервиса, написать общий `docker-compose.yml` и настроить сетевое взаимодействие между сервисами.

2 Ход работы

Контейнеризация выполнялась для микросервисной версии приложения, реализованной в рамках ЛР2. Проект представляет собой Gradle multi-module приложение на Kotlin и Spring Boot. В состав системы входят четыре бизнес-сервиса и две инфраструктурные зависимости: PostgreSQL и RabbitMQ.

2.1 Состав контейнеризуемого приложения

Сервис	Назначение	Порт
identity-service	Регистрация, авторизация, выдача JWT, получение данных текущего пользователя	8081
catalog-service	Каталог ресторанов, кухни, меню, столики, агрегированные рейтинги	8082
booking-service	Создание и просмотр бронирований, проверка доступности столика	8083
review-service	Создание и просмотр отзывов, публикация событий об отзывах	8084
postgres	Общая PostgreSQL база для локального запуска, схемы разделены по сервисам	5432
rabbitmq	Брокер сообщений для асинхронного взаимодействия	5672, 15672

2.2 Dockerfile для сервисов

Для каждого бизнес-сервиса был создан отдельный `Dockerfile`. Все файлы построены по одной схеме:

1. На первом этапе используется образ `eclipse-temurin:21-jdk`.
2. В контейнер копируется весь multi-module проект.
3. Gradle собирает `bootJar` только нужного сервиса.
4. На втором этапе используется более легкий runtime-образ `eclipse-temurin:21-jre`.
5. В итоговый образ копируется только готовый `jar`-файл сервиса.

Пример общей логики `Dockerfile`:

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY . .
RUN chmod +x ./gradlew \
    && ./gradlew --console=plain :booking-service:bootJar \
        -x liquibaseUpdate -x generateJooq --no-daemon

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /workspace/booking-service/build/libs/booking-service.jar
    app.jar
EXPOSE 8083
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

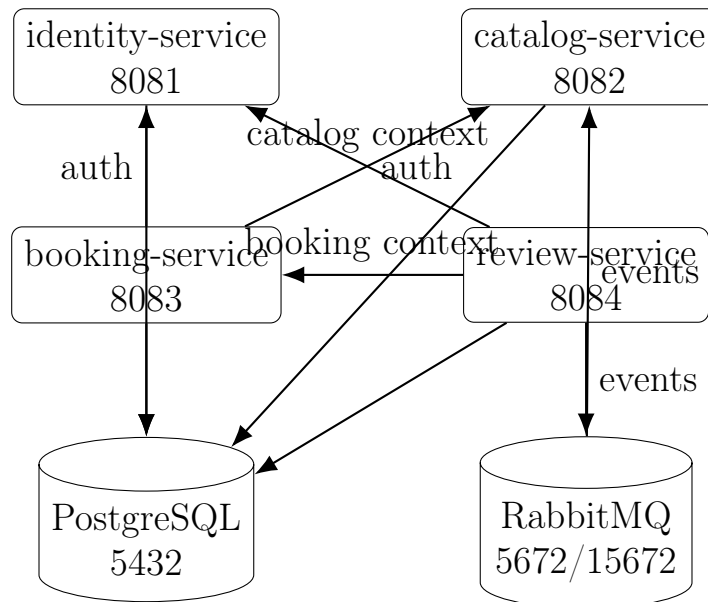
Генерация `jOOQ` и `Liquibase update` не запускаются внутри `Docker build`. Сгенерированные `jOOQ`-классы уже находятся в проекте, а миграции `Liquibase` применяются при старте каждого `Spring Boot` сервиса.

2.3 Оптимизация контекста сборки

Для уменьшения `Docker build context` добавлен файл `.dockerignore`. Из контекста исключены локальные Gradle-кэши, build-директории, временные файлы и PDF/TeX артефакты отчетов. Это ускоряет сборку образов и не тащит внутрь контейнеров лишние локальные файлы.

2.4 Общий `docker-compose.yml`

Для запуска всей системы добавлен общий `docker-compose.yml`. Он поднимает PostgreSQL, RabbitMQ и все четыре микросервиса в одной Docker-сети. Благодаря этому сервисы обращаются друг к другу не через `localhost`, а по DNS-именам контейнеров.



2.5 Сетевое взаимодействие

В Docker Compose все сервисы находятся в одной сети. Поэтому для меж-сервисных вызовов используются внутренние адреса:

Назначение	Адрес внутри Docker-сети
PostgreSQL	jdbc:postgresql://postgres:5432/storage_local
RabbitMQ	rabbitmq:5672
Identity Service	http://identity-service:8081
Catalog Service	http://catalog-service:8082
Booking Service	http://booking-service:8083

Например, **booking-service** получает адреса зависимых сервисов через переменные окружения:

```
SERVICES_CATALOG_BASE_URL: http://catalog-service:8082
SERVICES_IDENTITY_BASE_URL: http://identity-service:8081
```

review-service аналогично получает адреса **booking-service** и **identity-service**, а также параметры подключения к RabbitMQ.

2.6 Порядок старта

Для инфраструктуры настроены healthcheck-и. PostgreSQL считается готовым после успешного `pg_isready`, RabbitMQ — после успешного `rabbitmq-diagnostics ping`. Бизнес-сервисы запускаются после готовности нужной инфраструктуры.

Сервис	Зависимости при запуске
identity-service	postgres
catalog-service	postgres, rabbitmq
booking-service	postgres, identity-service, catalog-service
review-service	postgres, rabbitmq, identity-service, booking-service

2.7 Запуск и проверка

Для запуска всей системы используется команда:

```
docker-compose up --build -d
```

Если установлен новый Docker Compose plugin, можно использовать вариант:

```
docker compose up --build -d
```

После запуска доступны Swagger UI каждого сервиса:

Сервис	Swagger UI
identity-service	http://localhost:8081/swagger-ui/index.html
catalog-service	http://localhost:8082/swagger-ui/index.html
booking-service	http://localhost:8083/swagger-ui/index.html
review-service	http://localhost:8084/swagger-ui/index.html

Подключение к базе данных:

```
host: localhost
port: 5432
database: storage_local
username: storage
password: storage
```

RabbitMQ Management UI:

```
url: http://localhost:15672
username: storage
password: storage
```

Для остановки контейнеров используется команда:

```
docker-compose down
```

Для полной очистки данных PostgreSQL:

```
docker-compose down -v
```

3 Вывод

В рамках лабораторной работы была выполнена контейнеризация микросервисного backend-приложения. Для каждого сервиса создан отдельный Dockerfile, а общий `docker-compose.yml` поднимает всю систему вместе с PostgreSQL и RabbitMQ. Межсервисное взаимодействие настроено через внутренние DNS-имена Docker Compose, поэтому сервисы могут обращаться друг к другу внутри общей сети без ручной настройки адресов.

В результате приложение можно запускать одной командой, что упрощает демонстрацию на защите и приближает локальное окружение к реальному разворачиванию микросервисной системы.